Universidade do Porto

Faculdade de Engenharia

FEUP

# "Robô Seguidor de Linha"

Rui Martins
up202104636

Ivo Mendes
up202108509

Relatório do Trabalho Prático realizado no âmbito da Unidade Curricular
"Arquiteturas de Computação Embarcada", do
Mestrado em Engenharia Eletrotécnica e Computadores

Data de entrega: 30-01-2025

Declaramos que o presente trabalho/relatório é da nossa autoria e não foi utilizado previamente noutro curso ou unidade curricular, desta ou de outra instituição. As referências a outros autores (afirmações, ideias, pensamentos) respeitam escrupulosamente as regras da atribuição, e encontram-se devidamente indicadas no texto e nas referências bibliográficas, de acordo com as normas de referenciação. Temos consciência de que a prática de plágio e autoplágio constitui um ilícito académico.

In "Código Ético de Conduta Académica", art.14, Universidade do Porto.

# 1 Sumário

Este projeto envolveu o design, desenvolvimento e otimização de um robô capaz de seguir linhas e resolver labirintos (em forma de grelha) equipado com um Raspberry Pi Pico, múltiplos sensores e dois motores. Foram usadas técnicas de ajuste dinâmico de velocidade, controlo PID, calibração de sensores e mecanismos de desvio de obstáculos com o intuito de obter uma navegação mais precisa em trajetos complexos e irregulares de forma eficaz e autónoma.

# Índice

**1 Sumário** **1**

**2 Introdução** **2**

**3 Hardware** **3**
3.1 Componentes Usados . . . . . 3
3.2 Design do Circuito . . . . . 3

**4 Pista** **4**
4.1 Funcionalidades Principais . . . . . 4
4.1.1 Deteção de linha . . . . . 4
4.1.2 Implementação do Controlador PID . . . . . 5
4.1.3 Deteção de Obstáculos . . . . . 8

**5 Labirinto** **10**
5.1 Funcionalidade Principais . . . . . 10
5.1.1 Deteção do caminho mais curto - Algoritmo BFS . . . . . 10
5.1.2 Deteção de Obstáculos . . . . . 13
5.1.3 Movimentação entre interseções . . . . . 14
5.2 Modo de Operação . . . . . 17

**6 Desafios e Soluções** **18**

**7 Melhorias Futuras** **18**

**8 Conclusão** **18**

## 2 Introdução

A robótica representa um dos pilares da engenharia moderna, combinando hardware e software para resolver problemas complexos. Neste contexto, o presente trabalho teve como objetivo desenvolver um robô capaz de seguir uma linha preta, de percorrer labirintos estruturados em grelha, desviar de obstáculos e ajustar dinamicamente a sua velocidade e trajetória. Utilizando um Raspberry Pi Pico como microcontrolador, o sistema integra cinco sensores infravermelhos para deteção de linha, um sensor ToF (Time-of-Flight) para evitar colisões e um controlador PID para correção de erros.

A implementação de algoritmos como o Breadth-First Search (BFS) permitiu ao robô calcular o caminho mais curto em labirintos, enquanto técnicas de otimização de hardware, como o uso de multiplexadores, garantiram alguma eficiência no processamento. Este relatório detalha não apenas as escolhas de design e os desafios técnicos enfrentados, mas também demonstra como a interação entre software e hardware foi crucial para superar limitações práticas, como atrasos de leitura de sensores e falhas inesperadas em componentes.

A abordagem adotada seguiu uma metodologia iterativa, onde cada etapa de desenvolvimento foi validada através de testes empíricos.

# 3 Hardware

## 3.1 Componentes Usados

1. Microcontrolador: Raspberry Pi Pico.
2. Motores: Dois motores DC.
3. Sensores:
   - 5 sensores infravermelho (IR) para deteção da linha.
   - Sensor de distância VL53L0X Time-of-Flight (ToF) para deteção de obstáculos.
4. Multiplexador: CD74HC4067 para expansão das entradas analógicas.
5. Driver de motor: Dual H-Bridge para controlo da direção e velocidade dos motores.
6. Fonte de Alimentação: Pack de baterias externo.

## 3.2 Design do Circuito

O hardware do robô foi conectado da seguinte forma:

- **Sensores IR**: Conectados ao multiplexador, que os liga a uma das entradas analógicas do Raspberry Pi Pico.
- **Motores**: Controlados pelo driver de motor utilizando sinais PWM.
- **Sensor ToF**: Comunica-se via protocolo I2C.

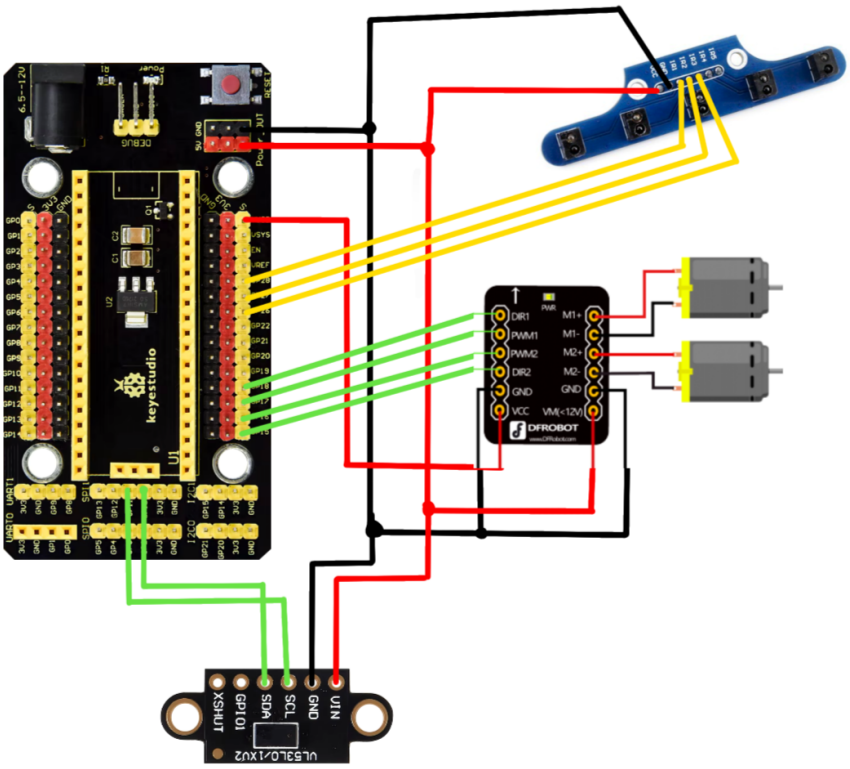


Figure 1: Conexões entre os componentes

# 4 Pista

## 4.1 Funcionalidades Principais

### 4.1.1 Deteção de linha

A primeira funcionalidade a merecer destaque é a deteção de linha. Como indicado no enunciado, o robô tem de ser capaz de identificar e guiar-se segundo uma linha preta num fundo branco, como ilustrado na Figura 2. Para tal, fazemos uso dos cinco sensores IR, bem como, de um método capaz de converter as leituras dos sensores em valores de erro passíveis de serem usados pelo controlador PID.

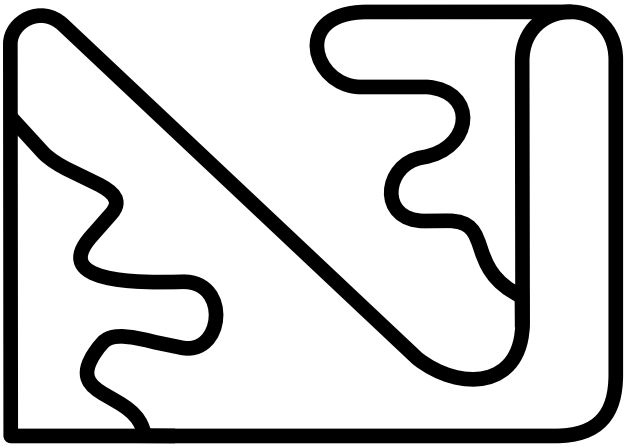

Figure 2: Mapa da pista.

Os sensores IR utilizados são capazes de registar valores num intervalo de 0 a 1023 e, sabendo que valores próximos de zero são zonas escuras e valores próximos de 1023 são zonas claras, somos capazes, através de um valor pré-definido de ***THRESHOLD***, identificar quais sensores estão sobre a linha. Para além disso, o robô é capaz de recalibrar os valores máximo e mínimo de leitura registados de forma a adaptar-se a diferentes intensidade de cor de linha, por exemplo, o que permite uma normaliza mais precisa dos valores registados.

```
void readLineSensors() {
  // Loop
  for (int i = 0; i < NUM_SENSORS; i++) {
    // Set the multiplexer to the current channel
    digitalWrite(MUX_S1, i & 0x01);        // Bit 0
    digitalWrite(MUX_S2, (i >> 1) & 0x01); // Bit 1
    digitalWrite(MUX_S3, (i >> 2) & 0x01); // Bit 2
    digitalWrite(MUX_S4, (i >> 3) & 0x01); // Bit 3

    delayMicroseconds(50); // Allow time for the signal to stabilize

    // Read the sensor value
    sensorValues[i] = analogRead(MUX_SIGNAL);

    // Recalibrate sensor values
    if(sensorValues[i] > sensorMaxValue) sensorMaxValue = sensorValues[i
        ];
    else if(sensorValues[i] < sensorMinValue) sensorMinValue =
        sensorValues[i];
  }

  // (...) THRESHOLD logic for line detection
```

```
  // Save last sensor values
  for (int i = 0; i < NUM_SENSORS; i++)
  {
    lastSensorValues[i] = sensorValues[i];
  }

}
```

As leituras dos sensores são guardadas num vetor ***sensorValues[5]*** de tamanho cinco. Estes valores são, posteriormente, utilizados num método ***calculateMediumSensor()***, como visto a seguir, que faz uso da média ponderada dos mesmos de forma a obter a posição da linha em relação aos cinco sensores.

```
float calculateMediumSensor() {
  float weightedSum = 0;
  float totalInvertedValues = 0;

  for (int i = 0; i < NUM_SENSORS; i++) {
    float invertedValue = sensorMaxValue - sensorValues[i];    //Black
        gets a higher value than white

    //Weighted sum and total inverted values
    weightedSum += invertedValue * sensorWeights[i];
    totalInvertedValues += invertedValue;
  }

  if (totalInvertedValues == 0) {
    return round((float)NUM_SENSORS/2); //Default to center position if
        no line is detected
  }

  return (weightedSum / totalInvertedValues) + round((float)NUM_SENSORS
      /2); //Average + Offsets posLine to the real position

}
```

### 4.1.2 Implementação do Controlador PID

Um controlador PID (Proporcional-Integral-Derivativo) é um mecanismo de controlo por retroação amplamente utilizado em sistemas de controlo automático para ajustar variáveis de processo, como velocidade ou posição. O seu objetivo é minimizar o erro entre um valor desejado (setpoint) e o valor medido da variável controlada, através da combinação de três ações distintas:

**1. Controlo Proporcional (P):**

Esta componente gera uma correção proporcional ao erro atual $(e(t))$. Permite reduzir o erro rapidamente, no entanto, pode provocar *erro residual* em regime permanente ou oscilações.

$$P = K_p e(t)$$

**2. Controlo Integral (I):**

Esta componente elimina o erro residual acumulado ao longo do tempo, integrando o erro passado. Tem como finalidade a remoção de desvios persistentes, mas pode introduzir oscilações caso seja mal dimensionado

$$I = K_i \int e(t)dt$$

**3. Controlo Derivativo (D):** Esta terceira componente antecipa futuros erros com base na taxa de variação do erro $\frac{de(t)}{dt}$. Amortece oscilações e melhora a estabilidade, mas é extremamente sensível a ruído existente no sistema.

$$D = K_d \frac{de(t)}{dt}$$

A soma das três componentes define a **saída do controlador**:

$$Saida = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt}$$

É através do valor ***posLine***, e de um valor pré-definido ***setpoint*** de valor 3.0, que calculamos o erro a ser utilizado pelo controlador PID. Foram testados diversos valores para os coeficientes tendo chegado aos seguintes:

- Coeficiente de proporcionalidade: **Kp** = 1.4;
- Coeficiente de integração: **Ki** = 0.025;
- Coeficiente de derivação: **Kd** = 0.025.

Juntamente com o controlo PID implementámos um método de controlo dinâmico de velocidade que alimenta diretamente o controlador. Este método é responsável pela deteção de curvas ou de retas, diminuindo ou aumentando a velocidade, respetivamente.

```
void calculatePWMAndDir(float posLine) {
  // Compute error
  float error = posLine - setpoint;

  // Compute time difference for integral and derivative
  unsigned long now = micros();
  float dt = (lastTime == 0) ? 0.02 : (now - lastTime) / 1000000.0; //
      Default to 20ms if lastTime=0
  lastTime = now;

  // Calculate integral (sum of errors over time)
  integral += error * dt;

  // Calculate derivative (rate of change of error)
  float derivative = (error - lastError) / dt;

  // Adjust motor speed dynamically based on error
  float dynamicSpeed = MOTOR_SPEED;
  if (abs(error) < 0.5) {
    dynamicSpeed = MOTOR_SPEED * map(abs(error), 0.0, 0.5, 1.0 +
        pastError, 1.0);
  } else if (abs(error) > 0.5) {
    dynamicSpeed = MOTOR_SPEED * 0.8;
  }

  //PID
  float correction = (Kp * dynamicSpeed) * error + (Ki * dynamicSpeed) *
      integral + (Kd * dynamicSpeed) * derivative;

  // Store the current error as the last error for the next iteration
```

```
  lastError = error;

  // Adjust motor speeds based on correction
  if (error > 0 + COMPENSATION) {
    if (correction >= dynamicSpeed) {
      motorDir[MOTOR1] = REVERSE;
      correction -= dynamicSpeed;
    } else {
      motorDir[MOTOR1] = FORWARD;
    }
    motorSpeed[MOTOR1] = abs(round(correction));
    motorSpeed[MOTOR2] = dynamicSpeed;
    motorDir[MOTOR2] = FORWARD;

    // Decrease dynamic speed coefficient
    pastError -= 0.1;
  } else if (error < 0 - COMPENSATION) {
    if (correction <= -dynamicSpeed) {
      motorDir[MOTOR2] = REVERSE;
      correction += dynamicSpeed;
    } else {
      motorDir[MOTOR2] = FORWARD;
    }
    motorSpeed[MOTOR2] = abs(round(correction));
    motorSpeed[MOTOR1] = dynamicSpeed;
    motorDir[MOTOR1] = FORWARD;

    // Decrease dynamic speed coefficient
    pastError -= 0.1;
  } else {
    motorDir[MOTOR1] = motorDir[MOTOR2] = FORWARD;
    motorSpeed[MOTOR1] = motorSpeed[MOTOR2] = dynamicSpeed;

    // Increase dynamic speed coefficient
    pastError += 0.1;
  }

  // Ensure motor speeds are within valid range
  motorSpeed[MOTOR1] = constrain(motorSpeed[MOTOR1], 0, 255);
  motorSpeed[MOTOR2] = constrain(motorSpeed[MOTOR2], 0, 255);
  pastError = constrain(pastError, 0, 1.5);
}
```

Deve ser, também, mencionada a capacidade do algoritmo ajustar a direção dos motores de acordo com o erro obtido. Considerando o facto do robô ter cinco sensores e o sensor do meio (sensor três) ser o sensor de referência, facilmente chegamos à conclusão que o erro, para o nosso caso, poderá assumir valores no intervalo de [-2 ; 2]. Subdividindo o intervalo do erro em quatro setores de amplitude igual a um obtemos:

- Extremo esquerdo: $erro \in [-2; -1]$;
- Centro esquerdo: $erro \in [-1; 0]$;
- Centro direito: $erro \in [0; 1]$;
- Extremo direito: $erro \in [1; 2]$;

Assim, e fazendo uso desta informação, sempre que o erro se encontre nos setores **extremo**, curva

acentuada, o motor que se encontre mais próximo da linha deve girar em sentido oposto de forma a rapidamente corrigir o sistema (direção do robô).

### 4.1.3 Deteção de Obstáculos

A deteção de obstáculos é feita com recurso às distâncias medidas e registadas pelo sensor VL53L0X e pelo método ***ostacleAvoidance()***. Aquando da deteção de um objeto, o robô entra num novo estado caracterizado por um comportamento de rotina, ou seja, o mesmo não faz uso de informações registadas pelos sensores para se deslocar e percecionar o meio que o rodeia. Segue apenas um caminho pré-definido.

Este novo estado faz uso de uma máquina de estados com o intuito de percorrer todas as funcionalidades pretendidas de forma sequencial.

```
void obstacleAvoidance(unsigned long currentMillis){
  switch (obstacleState)
  {
    case ALIGN:
      // Aligns the robot to get a precise 90 degree rotation
      if (alignRobot())
      {
        obstacleState = STOP;
        obstacleTimer = currentMillis;
      }
      break;

    case STOP:
      // Stops de robot
      motorSpeed[MOTOR1] = 0;
      motorSpeed[MOTOR2] = 0;
      if (currentMillis - obstacleTimer >= 500) {
        obstacleState = RIGHT;
        obstacleTimer = currentMillis;
      }
      break;

    case RIGHT:
      // Turns 90 degrees to the right
      if (!turning) {
        turning           = true;
        targetEncPosition = encPosition - TICKS_TURN_90;
      }

      if (encPosition <= targetEncPosition) {
        turning = false;
        motorSpeed[MOTOR1] = 0;
        motorSpeed[MOTOR2] = 0;

        obstacleState = FRONT;
        obstacleTimer = currentMillis;
      }
      else {
        motorSpeed[MOTOR1] = MOTOR_SPEED/2;
        motorDir[MOTOR1]   = REVERSE;

        motorSpeed[MOTOR2] = MOTOR_SPEED/2;
        motorDir[MOTOR2]   = FORWARD;
      }
      break;
```

```
    case FRONT:
      // Performs an half circle movement
      if (!turning) {
        turning            = true;
        targetEncPosition = encPosition + 7800;
      }

      if (encPosition >= targetEncPosition) {
        turning = false;

        obstacleState = END;
        obstacleTimer = currentMillis;
      }
      else {
        motorSpeed[MOTOR1] = MOTOR_SPEED;
        motorDir[MOTOR1]   = FORWARD;

        motorSpeed[MOTOR2] = MOTOR_SPEED/2;
        motorDir[MOTOR2]   = FORWARD;
      }
      break;

    case END:
      // Readjusts the robot before returning to the normal control
          state
      readLineSensors();
      if (currentMillis - obstacleTimer >= 1000) {
        start = true;
        obstacleState = STOP;
      } else {
        motorSpeed[MOTOR1] = MOTOR_SPEED/2;
        motorDir[MOTOR1]   = REVERSE;

        motorSpeed[MOTOR2] = MOTOR_SPEED/2;
        motorDir[MOTOR2]   = FORWARD;
      }
      break;
  }
}
```

**NOTA:** O sensor VL53L0X nem sempre deteta distâncias corretamente podendo levar à inicialização do processo de desvio de obstáculos de forma incorreta. Com o objetivo de corrigir o problema é usada uma variável que verifica se as últimas leituras por parte do sensor apresentam valores decrescentes entre si ou se ocorreu apenas um valor de pico.

# 5 Labirinto

## 5.1 Funcionalidade Principais

### 5.1.1 Deteção do caminho mais curto - Algoritmo BFS

O labirinto proposto é composto por uma **grelha retangular** com seis filas e cinco colunas, como ilustra a Figura 3. Tirando partindo desta informação, formulámos um algoritmo capaz de detetar o caminho mais rápido entre um ponto inicial (atual localização do robô) e o ponto final do labirinto.

Baseámo-nos no método **Breadth-First Search** (BFS) para resolver o algoritmo em causa. Este método é um algoritmo de procura em árvore, como ilustrado na Figura 4, que explora todos os vértices da mesma através de uma abordagem em camadas. Tipicamente este tipo de algoritmos recorre a filas para garantir que os nós mais próximos do ponto inicial são explorados primeiro, no entanto, encontrámos alguma dificuldade na implementação e uso de filas e, portanto, recorremos a vetores, o que nos obrigou a fazer alguns ajustes ao algoritmo original.

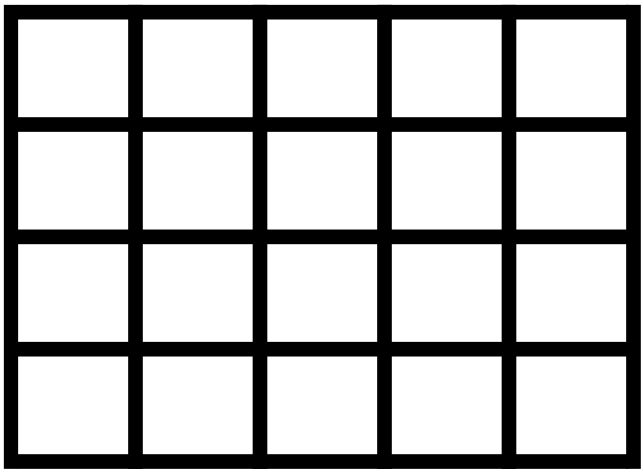

Figure 3: Mapa do labirinto.

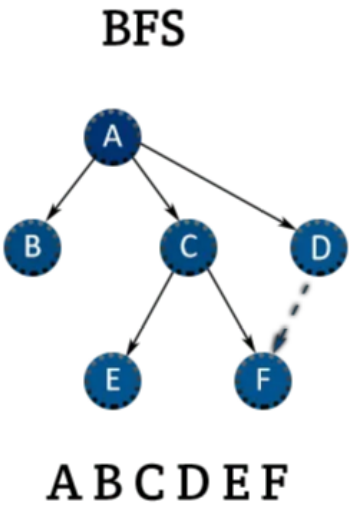


Figure 4: Algoritmo BFS e seu funcionamento.

Estruturalmente, recorremos a cinco métodos separados para implementar o algoritmo na sua totalidade. Esta abrodagem permite-nos tornar o código modular, versátil e receptivo a atualizações futuras se

tal se mostrar necessário.

Os três primeiros métodos formulados, e como referido anteriormente, são responsáveis por modificar o vetor ***queue*** e recriar o comportamento típico e único das filas. São eles: ***clearQueue()***; ***enqueue()***; ***dequeue()***.

```
void clearQueue() {
  queueFront = 0;
  queueBack  = 0;
  queueCount = 0;
}

bool enqueue(int rr, int cc) {
  // If the queue is full, return false
  if (queueCount >= MAX_QUEUE) {
    return false;
  }
  // Place the cell in queueArray at 'queueBack'
  queueArray[queueBack].r = rr;
  queueArray[queueBack].c = cc;
  // Advance 'queueBack' in a circular way
  queueBack = (queueBack + 1) % MAX_QUEUE;
  // Increment the count
  queueCount++;
  return true;
}

bool dequeue(int *outR, int *outC) {
  if (queueCount <= 0) {
    // Queue is empty
    return false;
  }
  // The front cell
  *outR = queueArray[queueFront].r;
  *outC = queueArray[queueFront].c;
  // Advance front index
  queueFront = (queueFront + 1) % MAX_QUEUE;
  queueCount--;
  return true;
}
```

O quarto método é responsável por executar a lógica proposta pelo método BFS. Este método percorre o labirinto, representado pelo vetor ***mazeGrid[6][5]***, verificando quais interseções já foram visitadas, quais estão obstruídas e possíveis caminhos para alcançar o ponto final do labirinto, retornando ***true*** caso haja um trajeto possível. O vetor ***mazeGrid*** é um vetor do tipo binário registando valores ***true*** para interseções livres e ***false*** para interseções obstruídas. O vetor ***visited[6][5]*** regista as interseções já visitadas pelo robô.

```
bool bfsComputePath(int startR, int startC, int goalR, int goalC) {
  // Reset visited + parent arrays
  for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLUMNS; c++) {
      visited[r][c]     = false;
      parentRow[r][c]   = -1;
      parentCol[r][c]   = -1;
    }
  }

  clearQueue();
```

```

  visited[startR][startC] = true;
  enqueue(startR, startC);

  // BFS loop
  while (queueCount > 0) {
    int r, c;
    if (!dequeue(&r, &c)) {
      break;
    }

    // If at the end of maze
    if (r == goalR && c == goalC) {
      return true;
    }

    // Check the 4 possible directions
    for (int i = 0; i < 4; i++) {
      int nr = r + directions[i][0];
      int nc = c + directions[i][1];

      // Check bounds
      if (nr >= 0 && nr < ROWS && nc >= 0 && nc < COLUMNS) {
        // If the cell is free and not visited
        if (mazeGrid[nr][nc] && !visited[nr][nc]) {
          visited[nr][nc] = true;
          parentRow[nr][nc] = r;
          parentCol[nr][nc] = c;
          enqueue(nr, nc);
        }
      }
    }
  }

  // If we exit the loop without finding the goal there is no path
  return false;
}
```

Como é visível na função anteriormente apresentada, existem dois vetores (***parent***) responsáveis por armazenar as interseções que antecederam a movimentação do robô pelo labirinto. Apesar de não serem utilizados diretamente e exclusivamente para esse propósito, por questões de praticidade, permitem-nos recriar o trajeto percorrido pelo robô. Na eventualidade de ser requerido que o robot retornarsse à posição inicial poderíamos recorrer a estes vetores, possibilitando uma implementação mais rápida e eficaz. Esta funcionalidade pretende demonstrar, novamente, o nosso comprometimento com a modularidade do código.

A quinta, e última função, possibilita-nos reconstruir o trajeto anteriormente encontrado. Após ser detetado a existência de, pelo menos, um possível trajeto, o mesmo é reconstruído e armazenado na variável ***path***.

```
void reconstructPath(int startR, int startC, int goalR, int goalC) {
  pathLength = 0;

  // If the goal wasn't visited, no path
  if (!visited[goalR][goalC]) {
    return;
  }

  // We'll store the path in reverse order first in a temporary buffer
```

```
  // (goal -> start), then we'll flip it.
  int tempRow[MAX_PATH];
  int tempCol[MAX_PATH];
  int count = 0;

  // Start from goal
  int r = goalR;
  int c = goalC;

  while (!(r == startR && c == startC)) {
    tempRow[count] = r;
    tempCol[count] = c;
    count++;

    // Jump to the parent cell
    int pr = parentRow[r][c];
    int pc = parentCol[r][c];
    r = pr;
    c = pc;
  }
  // Finally add the start node
  tempRow[count] = startR;
  tempCol[count] = startC;
  count++;

  // Now 'count' is the number of cells in the path
  // Reverse them into pathR[], pathC[] to get start->goal order
  pathLength = count;
  int idx = 0;
  for (int i = count - 1; i >= 0; i--) {
    pathR[idx] = tempRow[i];
    pathC[idx] = tempCol[i];
    idx++;
  }
}
```

### 5.1.2 Deteção de Obstáculos

Para a deteção de obstáculos, usamos uma lógica em tudo semelhante à da pista. Verificamos se as duas últimas leituras têm relação entre si, ou seja, se são decrescentes e, caso estejam abaixo do limite indicado, a função ***detectObstacle()*** retorna ***true***. A deteção de objetos desencadeia uma reconstrução do caminho previamente sugerido pelo algoritmo BFS.

```
bool detectObstacle(){
  if ( (prev_distance <= DIST_OBJECT * 2.5) && (distance <= DIST_OBJECT
      * 1.5) ){
    return true;
  }
  else return false;
}
```

A Figura 5 representa o momento de deteção de um objeto tornando o caminho inviável.

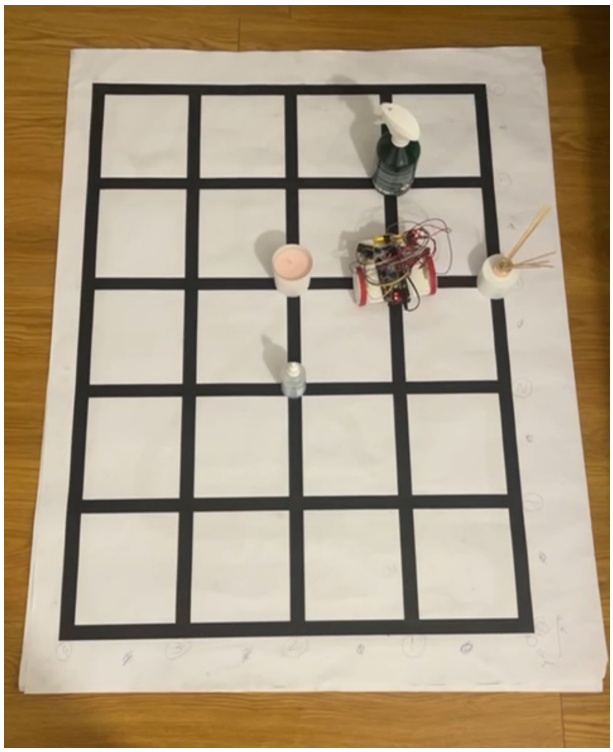

Figure 5: Obstáculo obstruindo o caminho.

### 5.1.3 Movimentação entre interseções

Após a descoberta do caminho mais curto até à posição final, é necessário deslocar o robô para a próximo interseção. Para tal temos de ter em consideração **dois fatores**:

- Direção atual e direção pretendida;
- Posição espacial no labirinto.

A direção atual (Norte, Sul, Este, Oeste) deve ser comparada com a direção pretendida para que o robô execute a rotação que permita alinhá-lo corretamente com o trajeto pretendido, ou seja, se o robô estiver voltado para Norte e a direção pretendida é Oeste, este deve girar noventa graus no sentido contrário aos ponteiros do relógio.

O método responsável por executar a rotação do robô é apresentado a seguir:

```
bool turn90(int side){ //"0" to go right and "1" to go left
 switch (side)
 {
 // Turn right
 case 0:{
   if (!turning) {
     turning            = true;
     targetEncPosition = encPosition - TICKS_TURN_90;

     motorSpeed[MOTOR1] = MOTOR_SPEED/2;
     motorDir[MOTOR1]   = REVERSE;

     motorSpeed[MOTOR2] = MOTOR_SPEED/2;
     motorDir[MOTOR2]   = FORWARD;

     motorControl();
   }
```

```
    if (encPosition <= targetEncPosition) {
      turning = false;
      motorSpeed[MOTOR1] = 0;
      motorSpeed[MOTOR2] = 0;

      motorControl();

      if(currDir == WEST) currDir = NORTH;
      else currDir++;

      return true;
    }
    break;
  }

  // Turn left
  case 1:{
    //(...) Same logic as the previous case but rotates to the other side
    break;
  }

  return false;
}
```

Verificada a direção correta do robô e devida rotação, o mesmo deve deslocar-se até à próxima interseção. Para tal, recorremos aos valores dos sensores das extremidades (sensores 1 e 5) e, mais precisamente, ao flanco descente dos mesmos. Caso o sensor, no ciclo, passado tenha detetado um valor inferior ao da THRESHOLD e o valor atual seja superior ao da THRESHOLD, detetamos a interseção e procedemos a um ajuste de posição e consequente paragem, reiniciando o ciclo.

A movimentação do robô recorre a uma versão mais simples do algoritmo anteriormente apresentado com o controlador PID. Devido à baixa velocidade do robô durante esta prova, não se justifica o uso de um algoritmo tão preciso.

A função responsável pela movimentação do robô entre interseções é a seguinte:

```
bool goToNextIntersection(int speed){
  // Read sensor values
  readLineSensors();

  if( intersection ) { // If intersection is detected
    switch (currDir)
    {
    case NORTH:
      currRow++;
      break;

    case SOUTH:
      currRow--;
      break;

    case EAST:
      currCol--;
      break;

    case WEST:
      currCol++;
      break;
```

```
    }

    motorDir[MOTOR1] = motorDir[MOTOR2] = FORWARD;
    motorSpeed[MOTOR1] = motorSpeed[MOTOR2] = 0;
    motorControl();

    intersection = false;
    atIntersection = false;

    return false;
  }
  else{ // If intersection is not detected keep moving
    float posLine = mazeCalculateMediumSensor();

    // Calculate error
    float error = posLine - setpoint;

    if (error > 0 + COMPENSATION) {
      motorSpeed[MOTOR1] = 0;
      motorDir[MOTOR1] = FORWARD;

      motorSpeed[MOTOR2] = speed;
      motorDir[MOTOR2] = FORWARD;
    } else if (error < 0 - COMPENSATION) {
      motorSpeed[MOTOR2] = 0;
      motorDir[MOTOR2] = FORWARD;

      motorSpeed[MOTOR1] = speed;
      motorDir[MOTOR1] = FORWARD;
    } else {
      motorDir[MOTOR1] = motorDir[MOTOR2] = FORWARD;
      motorSpeed[MOTOR1] = motorSpeed[MOTOR2] = speed;
    }

    motorControl();

    return true;
  }
}
```

Na Figura 6 podemos confirmar a movimentação anteriormente descrita após a deteção de obstáculos, ilustrada pela Figura 5.

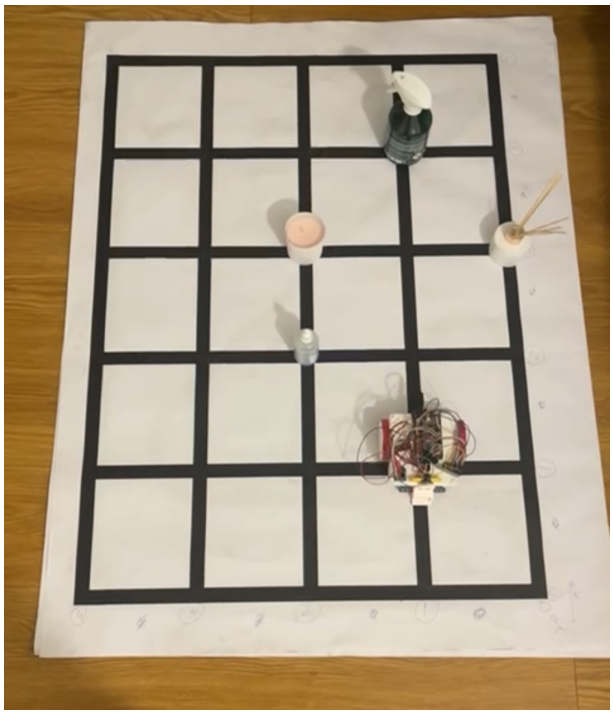

Figure 6: Inversão de marcha do robô após deteção de objetos.

## 5.2 Modo de Operação

Para resolver o labirinto, recorremos a uma máquina de estados composta por **nove estados**. Os mesmo encontrados enumerados a seguir:

```
enum{
  MAZE_STOP,
  MAZE_EXECUTE_PATH,
  MAZE_CALCULATE_SHORTEST_PATH,
  MAZE_ALIGN_CAR,
  MAZE_TURN_CAR,
  MAZE_DETECT_OBSTACLE,
  MAZE_MOVE_CAR,
  MAZE_ADJUST_CAR,
  MAZE_END
};
```

O estado ***MAZE CALCULATE SHORTEST PATH*** é considerado o estado inicial e é responsável pela deteção do caminho mais curto explicada em detalhe no Capítulo 5.1.1. Cada um dos estados assume um papel fundamental para a correta execução do teste do labirinto.

Após a execução do algoritmo, o código avança para o estado ***MAZE EXECUTE PATH***. Este estado marca o início da sequência de procedimentos para alcançar a próxima interseção/posição do caminho.

A sequência começa pelo alinhamento do robô com a linha preta (estado ***MAZE ALIGN CAR***), seguido pelo ajuste de rotação do robô se necessário (estado ***MAZE TURN CAR***). Concluída a rotação, o robô deve detetar a existência de qualquer tipo de obstáculo (estado ***MAZE DETECT OBSTACLE***) e se possível avançar (estado ***MAZE MOVE CAR***). Na eventualidade de existir um obstáculo no caminho, a sequência retorna ao início. Após a movimentação do robô e deteção da interseção, o robô deve ser ajustado (estado ***MAZE ADJUST CAR***)de forma a facilitar a execução da próxima iteração do ciclo. Antes de reiniciar o ciclo, o robô fica parado meio segundo (estado ***MAZE STOP***).

Quando o robô deteta que chegou ao destino/ponto final o mesmo para e não executa mais nenhuma função (estado ***MAZE END***).

# 6 Desafios e Soluções

O desenvolvimento do projeto enfrentou vários obstáculos técnicos que exigiram análise detalhada e ajustes iterativos. Um dos primeiros desafios surgiu com as leituras inconsistentes dos sensores, inicialmente atribuídas a interferências como variações de luminosidade. Para resolver este problema, implementou-se uma rotina de calibração automática executada a cada leitura de valores dos sensores. Para além disso, durante este processo, identificou-se que a função analogRead(), utilizada para leitura dos sensores, consumia recursos excessivos do microcontrolador criando alguns atrasados de execução.

Paralelamente, ocorreram inconsistências na direção de um dos motores, que não invertia o sentido de rotação conforme programado. Após verificação, descobriu-se que o problema estava relacionado alguns pinos da *motherboard* do microcontrolador, que deixaram de responder sem causa aparente. Infelizmente não foi possível corrigir estas falhas.

O uso de um multiplexador para expandir a capacidade de leitura de sensores também introduziu certas complicações. Verificou-se que o componente causava atrasos significativos no ciclo de leitura de valores dos sensores. Para contornar este limite, otimizou-se o loop de leitura, dividindo as leituras por fases e introduzindo microatrasos de estabilização entre cada operação, o que melhorou significativamente a eficiência e precisão do código.

Por fim, após uma queda durante os testes na pista, um dos motores começou a apresentar um barulho estranho quando girava no sentido oposto. Não foi possível corrigir esta falha por falta de tempo.

# 7 Melhorias Futuras

Embora o sistema atual funcione dentro dos parâmetros estabelecidos, existem várias otimizações que poderiam aumentar a sua eficiência e desempenho em cenários mais exigentes. Uma das principais áreas de melhoria reside no método de leitura dos sensores. Atualmente, os cinco sensores são lidos em cada ciclo de loop, mesmo quando apenas um subconjunto deles é relevante para decisões imediatas (por exemplo, em retas longas). Uma abordagem mais inteligente permitiria ler apenas os sensores necessários para cada situação, reduzindo o tempo de processamento.

A velocidade do robô no labirinto também apresenta margem para melhoria. Embora a configuração atual priorize a estabilidade, a utilização de algoritmos mais capazes ou a implementação de um controlador PID permitiriam acelerar o movimento em zonas seguras, como retas, desacelerando apenas em curvas ou intersecções.

Seria interessante, também, concluir a implementação do retorno do robô para a posição inicial do labirinto. Parte do processo e da lógica já se encontram implementados.

Por fim, a implementação de uma interface Wifi/Bluetooth (visto que foi usada uma Raspberry Pi Pico com módulo de Wifi) para comunicação com o robô à distância.

# 8 Conclusão

O projeto permitiu concluir com êxito a construção e desenvolvimento de um robô autónomo capaz de descodificar labirintos e seguir linhas pretas com precisão. A utilização de um controlador PID ajustável em tempo real garantiu uma resposta fluida a variações na trajetória, enquanto o algoritmo BFS assegurou eficiência na resolução de labirintos, mesmo na presença de obstáculos imprevistos.

Os desafios técnicos, como a inconsistência nas leituras dos sensores e as falhas de pinos do microcontrolador, não foram totalmente superados, no entanto, o robô demonstrou robustez suficiente para operar em ambientes controlados.

Este trabalho evidenciou o potencial da combinação entre algoritmos clássicos de procura e controlo proporcional-integral-derivativo em aplicações de robótica móvel. Futuras melhorias, como a implementação de comunicação sem fios, poderiam elevar o sistema a novos patamares de desempenho.